

- channelCount: The number of audio channels, which is 1 (mono).
- numberOfFrames: The number of frames in this chunk of data.
- type: Specifies that this part of the message is audio data.

Step 4: Processing Audio Data

To process the audio data received:

- Extract the samples array from the JSON data.
- Convert the samples into a suitable audio format if necessary, depending on your application's requirements.
- Store or transmit the audio data as needed for playback, analysis, or further processing.

Step 5: Sending Audio Data Back on WebSocket

1. Prepare the audio data you want to send back. Ensure it's in PCM linear 16-bit 8kHz format.
2. Structure the data in the required JSON format.
3. Send the JSON message via WebSocket.

```
{ "type": "media", "ucid": "111XXXXXXXXX71", "data": { "samples": [8,
8, 8, 8, 8, 8, 8, 8, 8, 8, 7, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8,
8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8,
8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8, 7, 7, 8, 8, 8],
"bitsPerSample": 16, "sampleRate": 8000, "channelCount": 1,
"numberOfFrames": 80, "type": "data" } }
```

Attaching Python Code using the websockets library. This example includes both receiving and sending audio data.

Clear the audio buffer on the WebSocket

```
{
  "command": "clearBuffer"
}
```

Disconnect call over the stream to handle normal ivr flow like to transfer call to human agents etc

```
{
  "command": "callDisconnect"
}
```